Let's see Jaeger in action: we again use a Docker Compose setup and configure Jaeger with ClickHouse as the span storage (listing 6.2). This time we're using a small-scale microservices setup with three `ho11y` instances wired up to simulate a frontend service that calls out to two backend services.

Listing 6.2 The relevant bits of Docker Compose file for Jaeger with ClickHouse setup

```
frontend:
    image: public.ecr.aws/mhausenblas/ho11y:stable
    ports:
    - 8765:8765
    environment:
    - "DOWNSTREAM0=http://backend0:8766"
    - "DOWNSTREAM1=http://backend1:8767"
    - "OTEL_RESOURCE_ATTRIB=frontend"
    - "OTEL_EXPORTER_OTLP_ENDPOINT=otel-collector:4317"
    - "HO11Y_INJECT_FAILURE=enabled"

  jaeger:
    image: jaegertracing/all-in-one:1.35
    command: [
      "--grpc-storage-plugin.binary=/etc/jaeger-clickhouse",
      "--grpc-storage-plugin.configuration-file=/etc/clickhouse-
     plugin.config"
    ]
    volumes:
    - ./jaeger-clickhouse:/etc/jaeger-clickhouse
    - ./clickhouse-plugin.config:/etc/clickhouse-plugin.config
    ports:
    - "9090:9090"
    - "16686:16686"
    - "14250:14250"
    environment:
    - "SPAN_STORAGE_TYPE=grpc-plugin"

  clickhouse:
    image: clickhouse/clickhouse-server
    ports:
    - "8123:8123"
    - "9000:9000"
```

The frontend microservice, using ho11y. It's configured to call the two backends and send traces to the OpenTelemetry collector.

The Jaeger all-in-one distribution configured with ClickHouse as the backend, using the gRPC-based plug-in mechanism

ClickHouse as the backend to store traces (and make them searchable)

We again use the OpenTelemetry collector to collect the traces from our `ho11y`-based microservices setup and ingest them into ClickHouse (via Jaeger). The respective OpenTelemetry collector configuration is shown in the following listing (see also chapter 4).

Listing 6.3 OpenTelemetry collector configuration for Jaeger with ClickHouse

```
receivers:
  otlp:
    protocols:
      grpc:
        endpoint: 0.0.0.0:4317
```